



# 设计模式

## 面向嵌入式软件开发



针对C语言及嵌入式软件开发的  
伴读教程，陪伴学习设计模式这一  
进阶知识点。

## 一、意图 (Intent)

在嵌入式系统中：

为硬件资源或关键对象提供一个受控访问入口。

核心：

- 所有访问必须经过代理层
- 代理层决定访问是否发生
- 调用方不关心底层细节

## 二、动机 (Motivation)

直接访问硬件的问题主要集中在三点：

### 1. 资源稀缺

- 总线共享 (I2C / SPI)
- Flash 擦写受限
- RAM 紧张

👉 需要统一控制访问

## 2. 状态复杂

- 未初始化 / 就绪 / 忙 / 错误

👉 状态不能分散在各处判断

---

## 3. 并发与成本

- 多任务竞争资源
- 外设访问有延迟 / 开销

👉 需要统一仲裁 + 控制访问频率

---

## 三、适用性 (Applicability)

适用于资源具备以下特征：

- 有状态 (init / busy / error)
- 有成本 (慢 / 限次数)
- 有竞争 (多任务访问)

常见需求：

- 状态检查
  - 延迟初始化
  - 缓存数据
  - 并发控制
- 

## 四、结构 (Structure)

业务层 → 代理层（控制入口） → 驱动/硬件

---

### 结构要点

#### 1. 接口稳定

业务层只面对统一接口

#### 2. 状态集中

代理层持有：

- 状态 (init / busy / error)
- 缓存
- 同步控制

👉 状态不再分散

### 3. 资源归属明确

初始化、缓存、释放统一在代理层

---

## 五、参与者 (Participants)

---

### 1. 接口

定义可操作能力（函数接口）

---

### 2. 真实对象

- 直接访问硬件
  - 不负责状态控制和并发
- 

### 3. 代理（核心）

负责：

- 状态管理
  - 访问控制
  - 缓存与同步
- 

### 4. 业务层

只调用接口，不接触硬件

---

## 六、协作 (Collaborations)

---

基本流程：

1. 业务层调用接口
  2. 代理层判断状态（init / busy 等）
  3. 决定是否访问真实对象
  4. 返回结果（可能来自缓存）
- 

👉 关键点：代理决定是否触发真实访问

---

## 七、效果 (Consequences)

---

### ✓ 优点

- 状态集中
  - 资源可控
  - 并发安全
  - 降低耦合
- 

### ✗ 代价

- 增加一层调用
  - 占用额外内存 (状态 / 缓存)
  - 状态管理复杂度上升
- 

## 八、实现要点 (Implementation)

---

### 1. 状态集中

- init\_flag
  - busy\_flag
  - error\_flag
  - cache\_valid
- 

### 2. 内存控制

- 优先静态分配
  - 控制缓存大小
- 

### 3. 访问策略

明确：

- 是否缓存
  - 是否阻塞
  - 是否允许并发
- 

## 九、典型应用

---

- 传感器访问 (缓存数据)
- Flash / EEPROM (控制访问频率)
- 通信模块 (状态管理 + 节流)
- 多任务共享外设 (统一加锁)

---

# 总结

---

代理模式 = 给硬件访问加一个“统一控制入口”，把状态、资源、并发全部收口管理。